



university of
groningen

Quantitative text analysis – Essex Summer School

Supervised classification, evaluation, and LLM-assisted labels

dr. Martijn Schoonvelde

University of Groningen

Today's class

- An overview of supervised classification
- Human coding, LLM-assisted coding, and validation
- Lab 5: Training and evaluating classifiers with Corbyn and May speeches
- AOB week 1

Jeremy Corbyn and Theresa May



Jeremy Corbyn

Labour leader, 2015–2020



Theresa May

Conservative leader, 2016–2019

- We'll focus on House of Commons speeches by **Jeremy Corbyn** and **Theresa May** during their time as party leader.
- How well can we **predict whether a speech was delivered by Corbyn or May?**

From texts to predictions

Supervised classification turns your research question into a prediction problem.

texts → features → model → predicted label

Step	
Texts	House of Commons speech contributions
Features	words, TF-IDF weights, or other representations
Label	Corbyn or May
Model	learns which textual patterns predict each label
Evaluation	checks predictions against held-out speaker labels

What could the classifier learn?

Corbyn example

"... tax credits and the devastation caused to many families ..."

Jeremy Corbyn. Engagements, 21 October 2015.

May example

"... if we are going to talk about the economic record, let us get the facts straight ..."

Theresa May. Engagements, 13 July 2016.

The model does not understand leadership or style directly. It learns from **regularities in labelled examples**: which words and phrases tend to appear more often in one speaker's speeches than another's.

A definition of supervised machine learning

“A form of machine learning where we aim to predict a variable, that, for at least part of our data, is known” (Van Atteveldt, 2022: p. 114)

- We estimate a model based on some complete data, and then use the model to predict the expected outcome for some new cases, for which **we do not know the outcome yet**

In the context of supervised text analysis, we rely on **textual data** to make our predictions

- In the conventional machine learning paradigm, the computer often requires labelled data that map input features (often BoW representations) to output labels (e.g., sentiment, topics, hate speech, party)
- Recent developments in text as data (weak supervision and transfer learning) reduce the amount of high quality labelled data required (more on this later in the course)

Workflow and terminology

1. Categorize a set of documents **by hand** or use trusted metadata – create a **labelled dataset**
2. Divide categorized documents in a **training set** and a **test set**
3. Use a supervised algorithm to **learn the relationship** between the known labels and the features in **the training set**
4. Evaluate the classifier based on whether it **correctly categorizes** documents in **the test set**
5. Use the classifier to categorize **unseen texts** (not part of the training and the test set)

Adapted from Van Atteveldt *et al.* (2022)

What makes a label trustworthy?

The model can only learn from the labels we give it.

Label source	Useful when. . .	Main risk
Human coding	the concept requires judgment	coders disagree or drift
Metadata	the label already exists, as with speaker	metadata is only a proxy for the concept
LLM label	the concept is spelled out in a codebook	errors look fluent and systematic

NB: Supervised learning validates a mapping from **textual evidence** to **labels**; it does not make the labels valid by itself.

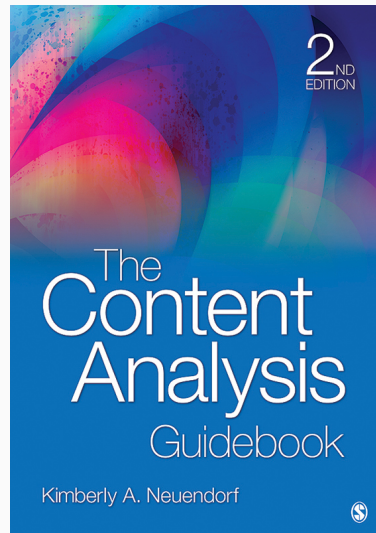
Creating a training set using hand coding

Why hand-code? To create a high-quality, labelled dataset for training and validating supervised machine learning models.

Steps (Grimmer, Roberts, and Stewart, 2022):

1. Define a codebook with clear category definitions.
2. Select and train coders.
3. Sample documents with diversity and balance in mind.
4. Coordinate coding and answer questions.
5. Check reliability using metrics such as Cohen's kappa or Krippendorff's alpha.
6. Address coder drift through calibration.

Source: Neuendorf (2017), *The Content Analysis Guidebook*



Evaluating a classifier: accuracy

The full leader-period dataset is imbalanced.

- Theresa May: 6,997 speeches.
- Jeremy Corbyn: 1,454 speeches.

Suppose we ignore the text and always predict “May”.

- We would achieve 82.8% accuracy.
- That sounds excellent, but it completely fails to identify Corbyn speeches.
- Accuracy needs context: always compare against the class distribution and the research goal.

Evaluating a classifier: confusion matrix

A **confusion matrix** is a table that summarizes the **performance of a classifier**.

		True Label	
		Positive	Negative
Prediction	Positive	True Positive	False Positive (Type I error)
	Negative	False Negative (Type II error)	True Negative

Performance metrics

- **Precision:** This is the ratio of correctly predicted positive observations to the total predicted positives. High precision means that an algorithm returned more relevant results than irrelevant ones.
- **Recall:** This is the ratio of correctly predicted positive observations to all observations in the positive class. High recall means that an algorithm returned most of the relevant results.
- **F1 Score:** This is the weighted average of Precision and Recall. High F1 score means that both recall and precision are high.

Evaluating a classifier: confusion matrix

- In Lab 5, a Naive Bayes model gives the following held-out predictions on a **balanced** Corbyn-May dataset.
- Here, the positive class is **Corbyn**.

		Reality	
		Corbyn	May
Prediction	Corbyn	411	25
	May	38	399

Evaluating a classifier: confusion matrix

		Reality	
		Corbyn	May
Prediction	Corbyn	411 (TP)	25 (FP)
	May	38 (FN)	399 (TN)

Evaluating a supervised classifier: precision and recall

Precision, recall and the F1 score are frequently used to **assess classification performance**:

- Precision: **$TP / (TP + FP)$**
 - *When we predict Corbyn, how often are we right?*
- Recall: **$TP / (TP + FN)$**
 - *How many actual Corbyn speeches do we find?*
- F1 score is a harmonic mean of precision and recall **$2 * (Precision * Recall) / (Precision + Recall)$** .

Evaluating supervised methods: precision and recall

- Precision: $\text{TP} / (\text{TP} + \text{FP})$
 - $\rightarrow 411 / (411 + 25) = 0.943$
- Recall: $\text{TP} / (\text{TP} + \text{FN})$
 - $\rightarrow 411 / (411 + 38) = 0.915$
- F1 score is a harmonic mean of precision and recall $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$.
 - $\rightarrow 2 * (0.943 * 0.915) / (0.943 + 0.915) = 0.929$

How to find the best model?

Precision, recall and F1 scores are useful, but it is up to us which classifier to choose. As always, our decision depends on our **research goals**.

- If we are determined not to miss Corbyn speeches, we may prioritise **recall**
- If we want high confidence that predicted Corbyn speeches are actually Corbyn speeches, we may prioritise **precision**

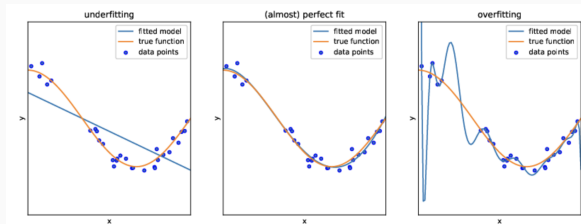
F1 scores **compromise between both**, but we do not want **precision and recall to diverge too much**

- For classifiers that also give us a prediction probabilities, we can also display a ROC curve, a plot that displays true positive against false positive at different probability thresholds

Generalisation and overfitting

- **Generalisation:** a classifier learns to correctly predict labels from given inputs not only in previously seen samples but **also in previously unseen samples**
- **Overfitting:** a classifier learns to correctly predict labels from given inputs in previously seen samples but **fails to do so on previously unseen samples**

Source: Van Atteveldt, Trilling, Arcila Calderón (2022)

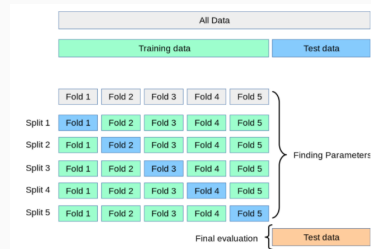


Cross validation

If we test the performance of multiple classifiers on our test data, we run the risk of overfitting our model to the test data

- One approach to address this issue is called **k-fold cross validation** where we separate our data into k separate training and test sets, and average our model results.
- If our classifier **generalizes well**, we should expect that our metric of choice is **similar in all folds**

Source: <https://stats.stackexchange.com/>



Naive Bayes classifier

Intuition: If we observe terms such as “tax credits”, “Prime Minister”, or “the Government” in a speech, how likely is the speech to come from Corbyn or May?

Evidence in a speech	What the model asks
tax_credits, austerity, families	Are these more common in Corbyn-labelled training speeches?
government, deal, facts	Are these more common in May-labelled training speeches?
very common words	Do they add little information because both speakers use them?

The classifier combines a **prior** guess about speaker frequency with the **word evidence** in the speech.

Naive Bayes classifier

- Use Bayes' rule to predict that a document has a certain label.
 - It can be applied when we have two labels or many labels.
- Naïve assumption – all features are independent from each other, as if they are random draws from a vocabulary

Bayes' theorem:

$$P(\text{label} \mid \text{features}) = \frac{P(\text{features} \mid \text{label}) P(\text{label})}{P(\text{features})}$$

NB: thanks to Stefan Müller for the examples in the next few slides.

Naive Bayes classifier

1. Estimate the frequency of each class in the training data (**prior**).
2. Estimate how common each feature is within each class.
3. Combine the prior and feature evidence to estimate the probability of each class (**posterior**).
4. Which posterior is highest?

Naive Bayes classifier

Advantages

- Simple, fast, effective
- Relatively small training set required if classes are not too imbalanced; easy to obtain probabilities

Disadvantages

- Assumption of conditional independence is problematic
 - In a document about football we expect words like **ball**, **victory**, **league** to cluster
- If a feature is not in the training set, it is disregarded for classification

Naive Bayes in quanteda

```
speaker_classifier_nb <- textmodel_nb(  
  train_dfm,  
  y = docvars(train_dfm, "speaker_label"),  
  prior = "docfreq"  
)  
  
matched_test_dfm <- dfm_match(  
  test_dfm,  
  features = featnames(train_dfm)  
)  
  
pred_speaker_nb <- predict(  
  speaker_classifier_nb,  
  newdata = matched_test_dfm,  
  type = "class"  
)
```

Naive Bayes predictions in Lab 5

```
tab_class_nb <- table(  
  predicted_speaker = pred_speaker_nb,  
  actual_speaker = docvars(test_dfm, "speaker_label")  
)  
  
tab_class_nb  
  
caret::confusionMatrix(tab_class_nb, mode = "prec_recall")  
  
Accuracy : 0.928  
Precision: 0.943  
Recall    : 0.915  
F1        : 0.929
```

The high performance is useful, but it requires interpretation. The model may learn speaker style, but also institutional role, time period, Brexit topics, etc.

Worked example: precision, recall, and F1

- Positive class: Corbyn
- True positives: 411
- False positives: 25
- False negatives: 38

$$\text{Precision} = \frac{411}{411 + 25} = 0.943$$

$$\text{Recall} = \frac{411}{411 + 38} = 0.915$$

$$\text{F1} = 2 \times \frac{0.943 \times 0.915}{0.943 + 0.915} = 0.929$$

Other supervised classifiers

Model family	Intuition
Naive Bayes	words provide probabilistic evidence for each class
Logistic regression	features receive weights that push predictions toward one class
Support Vector Machine	finds a boundary that separates classes well
Transformer/LLM classifier	uses contextual representations learned from large corpora

NB: “We will happily choose better features over better models every time” (Grimmer, Roberts, and Stewart, p. 185).

For a useful R introduction, see Hvitfeldt and Silge, *Supervised Machine Learning for Text Analysis in R*: <https://smlltar.com/>.

How to obtain high-quality labelled data?

Traditional options:

- Recruit and train expert coders.
- Use crowd-workers on platforms such as Amazon Mechanical Turk (MTurk) (Benoit *et al.*, 2016).
- Use trusted metadata when it exists



Figure 1: *ChatGPT* zero-shot text annotation performance, compared to MTurk and trained annotators. *ChatGPT*'s accuracy outperforms that of MTurk for four of the five tasks. *ChatGPT*'s inter-coder agreement outperforms that of both MTurk and trained annotators in all tasks. Accuracy means agreement with the trained annotators.

Source: Gilardi *et al.* (2023)

Recent work asks whether LLMs can act as **coders** in supervised text analysis workflows:

- Gilardi *et al.* (2023) find that GPT-3 can measure topics, stances, opinions, and frames in tweets.
- Ziems *et al.* (2023) find that LLMs perform reasonably well across a wide range of social-science annotation tasks.

New ways to reduce hand coding

Approach	Main idea	What the model is doing
Weak supervision	use imperfect signals as labels	combines dictionaries, rules, metadata, or model outputs
Transfer learning	reuse knowledge from a model trained elsewhere	adapts a pretrained model to a new classification task
LLM-assisted coding	ask an LLM to apply a codebook	produces document-level labels, explanations, or uncertainty notes

NB: transfer learning is about where the model gets its prior knowledge; LLM-assisted coding is about using a model as part of the annotation workflow.

Transfer learning and LLM-assisted coding

These categories can overlap, but they play different roles in a QTA workflow.

- In **transfer learning**, the model is usually the classifier or representation engine.
- In **LLM-assisted coding**, the model acts more like a coder applying instructions.
- Both reuse pretrained knowledge but they enter the research design differently.

Validating LLM-assisted coding

LLMs can make coding faster, but an LLM-coded dataset is still a **measurement instrument**.

Validation question	Why it matters
Does the output follow the codebook?	labels must be explicit and comparable
Is the output structured?	invalid labels and missing fields should be inspectable
Does a human-coded sample agree?	gives a benchmark for precision, recall, and bias
Are errors systematic?	failures may cluster by party, topic, time, speaker, or agenda
Is provenance recorded?	model, prompt, codebook, date, and settings are part of the method